UAP Coin: Viva and Coding Test Guide

20 viva questions with simple answers, the bugs examiners may find, and 10 coding tasks with working code

How to use this guide

1. Read **Part 1** once so you understand the project in plain words.
2. For each question in **Part 3**, learn the green **"Say this in the viva"** line first, then read the explanation under it.
3. Read **Part 4** (bugs) because examiners love asking "what is wrong with this code?".
4. Practise the tasks in **Part 5** by typing them yourself, without copying.

Part 1. The project in one minute

UAP Coin is a small Bitcoin-style blockchain written in Python. It can do four things: (1) **create a wallet** (private key, public key, address), (2) **build a block** that contains one special "reward" transaction, (3) **mine** the block by finding a hash that starts with four zeros, and (4) **save** the block in a file. It does not yet have a network, normal user-to-user transactions or chain checking. Those are exactly the things the coding test may ask you to add.

1. Wallet	2. Build block	3. Mine	4. Save
make keys and address	header + coinbase transaction	try nonces until hash starts with 0000	write block to Data/uapcoin-blockchain

What each file does

File	What it does (in simple words)
`main.py`	Starts the program: creates the blockchain object and adds two blocks, then prints the chain.
`config.py`	Settings: version, mining target (`"0000"`), coin name, block reward (50), and `ONE_COIN` = 100,000,000 poisha.
`Client/account.py`	The wallet. Makes a random private key, public key and Base58 address.
`Core/blockchain.py`	The main class. Creates the genesis block, adds new blocks, reads and writes the chain.
`Core/blockheader.py`	The block header and the `mine()` function (proof of work).
`Core/block.py`	A simple container: height, size, header, transactions, tx count.
`Core/tx.py`	Transactions: coinbase (reward) transaction, inputs, outputs, serialization, TxID.
`Core/script.py`	Locking scripts. Builds the P2PKH script used in transaction outputs.
`Core/Database/database.py`	Saves and loads the chain as one JSON line in `Data/uapcoin-blockchain`.
`Util/util.py`	Helpers: SHA-256, hash160, Base58 encode/decode, varint, byte-order conversion.
`Core/EllepticCurve/`	Maths for secp256k1: field elements, points, signing, verifying.

To run it: install the requirement `pycryptodome` (needed for RIPEMD-160), open a terminal in the `Blockchain` folder and run `python main.py`. The database path `Data` is relative, so it is created wherever you start Python. Start it from the `Blockchain` folder so it finds the existing file.

Part 2. Key words (learn these first)

Word	Simple meaning
Hash	A fingerprint of data. Same data gives the same hash. Change one letter and the hash changes completely. You cannot go backwards from hash to data.
SHA-256	The hash function used here. Output is 64 hex characters (256 bits).
Block	A box that holds transactions plus a header.
Blockchain	Blocks linked in order. Each block stores the hash of the previous one, like a chain.
Nonce	"Number used once." The number a miner keeps changing to get a lucky hash.
Mining / Proof of work	Trying many nonces until the hash meets the rule (starts with 0000). Hard to find, easy to check.
Transaction	A record that moves coins: inputs (coins being spent) and outputs (who receives them).
UTXO	Unspent Transaction Output. A coin "note" that has been received and not yet spent.
Coinbase transaction	The first transaction in a block. It creates new coins as the miner's reward.
Merkle root	One hash that represents all transactions in a block.
Private key	A secret 256-bit number. Whoever knows it controls the coins. Never share it.
Public key	A point on a curve calculated from the private key. Safe to share.
Address	A short, readable form of the (hashed) public key. People send coins to it.
Poisha	The smallest unit of UAP Coin. 1 UAP Coin = 100,000,000 poisha.

Part 3. The 20 viva questions with answers

A. Blocks, mining and the chain

Q1 What is a block, and what does each field of the BlockHeader do?

> **Say this in the viva:** A block is a container of transactions. Its header holds the data that links it to the previous block and proves work was done.

Explained simply

Think of a page in a ledger book. The **header** is the label on top of the page; the **transactions** are the entries.

Header field	What it means
`version`	Version of the block format (here 1.0).
`prev_block_hash`	Hash of the previous block. This is what makes it a chain.
`merkle_root`	One hash that stands for all transactions in this block.
`timestamp`	When the block was created (Unix time in seconds).
`bits`	Difficulty setting. Here it is only a placeholder (`"ffff0000"`).
`nonce`	The number the miner changes to find a valid hash.
`blockHash`	The final hash of all the fields above. It must start with 0000.

Q2 How does mining work in your code? What happens if TARGET is changed to "00000"?

> **Say this in the viva:** Mining means changing the nonce and hashing the header again and again until the hash starts with the target zeros. It is hard to find but very easy to verify.

Explained simply

The hash looks random, so there is no shortcut. The miner just guesses. In `mine()`: join the header fields into text, hash it, check if it starts with `"0000"`, and if not add 1 to the nonce and try again.

Each hex digit is 0 with chance 1 in 16. So four zeros need about 16^4 = **65,536** tries on average. Five zeros need 16^5 = **1,048,576** tries, so it becomes **16 times slower**.

Example from your saved chain: block 1 was found at nonce 26,737 and its hash starts with `0000cf61...`. Anyone can check it with one hash calculation, but finding it took thousands of tries.

Q3 What is the genesis block? Why is prev_block_hash all zeros?

> **Say this in the viva:** The genesis block is the very first block. It has no previous block, so its prev_block_hash is a fixed value of 64 zeros (ZERO_HASH).

Explained simply

It is created in `create_genesis_block()` when no database file exists. Every later block takes the hash of the block before it.

Small detail: in your code the genesis block has height 1. In Bitcoin the genesis block has height 0. Mention this if asked, it shows you read your code.

Q4 What is a coinbase transaction? Why prev_tx = 000... and prev_index = 0xFFFFFFFF?

Say this in the viva: A coinbase transaction is the first transaction of every block. It creates new coins as the miner's reward, so it has no real input.

Explained simply

Normal transactions spend an earlier output. The coinbase spends nothing, because the coins are brand new. To show this, its input is marked with **prev_tx = 64 zeros** and **prev_index = 0xFFFFFFFF**. The function `is_coinbase()` checks exactly these two values.

Its `script_sig` stores the **block height**, so every block's coinbase has a different TxID. The output pays `CURRENT_REWARD * ONE_COIN` (50 coins) to the miner address.

Q5 What is ONE_COIN = 100,000,000? Why integers instead of decimals?

Say this in the viva: 1 UAP Coin equals 100,000,000 poisha, the smallest unit. Integers are used because decimals (floats) cannot store money exactly.

Explained simply

In Python, `0.1 + 0.2` gives `0.30000000000000004`. For money, tiny errors are not allowed. So all amounts are stored in poisha as whole numbers and only converted to coins when shown: `50 * 100_000_000 = 5,000,000,000` poisha. Bitcoin does the same with satoshis.

Q6 What is the UTXO model? How is it different from an account balance?

Say this in the viva: In the UTXO model, coins exist as unspent outputs of earlier transactions. To pay, you spend whole outputs and get change back. A balance is the sum of your unspent outputs.

Explained simply

Cash note example: you hold one 500 taka note (one UTXO). You buy something for 200. You hand over the whole note and get 300 back. In a transaction: **input** = your 500 output; **outputs** = 200 to the shop + 300 change to yourself. The old 500 output is now spent.

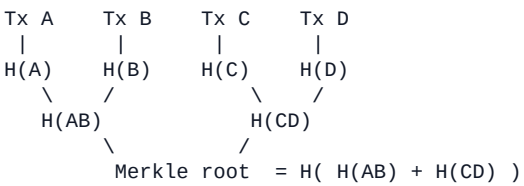
In an **account model** (like a bank), the system keeps one balance number per person and just subtracts and adds.

Q7 What is a Merkle root? Why is it empty in your code?

Say this in the viva: A Merkle root is a single hash that represents all transactions in a block. It is empty here because the feature is not implemented yet (only one transaction exists).

Explained simply

Hash each transaction, then hash pairs together, again and again, until one hash is left:



If any transaction changes, the root changes, so the block hash changes too. It also lets light wallets prove a transaction is in a block with only a few hashes. In your code `merkle_root = ''` is a placeholder (task 1 fixes it).

Q8 How does the chain detect tampering? What if I edit block 3?

Say this in the viva: Editing block 3 changes its hash, which no longer matches the prev_block_hash saved in block 4, so the chain breaks. To hide the change you would have to re-mine block 3 and every block after it.

Explained simply

Each block stores the previous block's hash, like a seal. Change anything inside block 3 (even one amount) and its hash changes. Block 4 still holds the old hash, so block 4 now points to a block that no longer exists. Re-mining blocks costs real work, so tampering is practically impossible on a big network.

Be honest in the viva: your project does not yet have a function that checks the chain. Task 3 (`is_chain_valid()`) adds it.

B. Keys, addresses and cryptography

Q9 Walk through Account.createKeys() step by step.

Say this in the viva: Random private key, multiply by generator point G to get the public key, compress it, hash160 it, add a prefix and checksum, then Base58 encode it.

Explained simply

Step	What the code does	Why
1	`secrets.randbits(256)`	Make a random secret number (the private key).
2	`privateKey * G`	Multiply the generator point on the secp256k1 curve. Result is the public key point (x, y).
3	Prefix `02` or `03` + 32 bytes of x	Compressed public key: the prefix says if y is even or odd, so y can be rebuilt.
4	`hash160_bytes(...)`	SHA-256 then RIPEMD-160. Gives a short 20-byte value.
5	Add `0x00` in front	Mainnet prefix (21 bytes now).
6	Add first 4 bytes of SHA-256 of that	Checksum, to catch typing mistakes (25 bytes now).
7	Base58 encode	Short, readable address that starts with 1.

The last line of the function, `self.publicKey == decode_base58(self.PublicAddress)`, checks that decoding the address gives back the same 20-byte hash. It should print True.

Q10 What is secp256k1? What are Gx, Gy, P and N? Why is the public key privateKey * G?

Say this in the viva: secp256k1 is the elliptic curve y squared = x cubed + 7 used by Bitcoin. G is the agreed starting point, and the public key is G added to itself private-key times.

Explained simply

P = the big prime number the curve works under ($2^{256} - 2^{32} - 977$). **G = (Gx, Gy)** = a fixed public starting point. **N** = how many points the group made by G has; private keys must be smaller than N.

Going forward (key times G) is fast. Going backward (finding the private key from the public point) is practically impossible. This one-way property is what lets you show your public key without giving away your private key.

Q11 Why hash160 (SHA-256 then RIPEMD-160), a checksum, and Base58?

Say this in the viva: hash160 makes the address short and hides the raw public key. The checksum catches typing mistakes. Base58 removes confusing characters.

Explained simply

hash160: shorter (20 bytes) and adds another hash layer in front of the public key.

Checksum: the first 4 bytes of a double hash. If someone mistypes one character, the checksum will not match and `decode_base58` raises an error, so coins are not sent to a wrong address by accident.

Base58: like Base64 but without `0 O I l` (they look alike) and without `+ /`, so addresses are easy to read and copy.

Q12 What does the 0x00 prefix mean? Why does the address start with 1?

> **Say this in the viva:** 0x00 is the mainnet version byte. In Base58, every leading zero byte is written as the character 1.

Explained simply

The prefix tells software which network the address belongs to (mainnet vs testnet). The Base58 code counts leading zero bytes and puts one `"1"` for each. Since the prefix byte is 0x00, the address begins with 1.

Q13 What range must a private key be in? Is randbits(256) safe enough?

> **Say this in the viva:** A private key must satisfy 1 <= k < N. randbits(256) can in theory give 0 or a number above N, so a safer call is secrets.randbelow(N - 1) + 1.

Explained simply

The chance of a bad value is tiny (about 1 in 2^{128}), so it works in practice, but a correct program should still enforce the range:

```
N = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
self.privateKey = secrets.randbelow(N - 1) + 1     # always 1 <= k < N
```

C. Transactions and scripts

Q14 What is P2PKH? Explain OP_DUP OP_HASH160 <hash> OP_EQUALVERIFY OP_CHECKSIG.

> **Say this in the viva:** P2PKH (Pay to Public Key Hash) locks coins to the hash of a public key. To spend them you show the public key and a valid signature.

Explained simply

Lock and key idea: the output is a lock that only opens for the owner of a certain public key. The spender gives a signature and the public key. The script then runs on a stack, step by step:

Opcode	What it does
`OP_DUP (0x76)`	Copies the public key on top of the stack.
`OP_HASH160 (0xA9)`	Hashes that copy with SHA-256 + RIPEMD-160.
`<pubKeyHash>`	Pushes the hash saved in the output (the owner's address hash).
`OP_EQUALVERIFY (0x88)`	Checks the two hashes are equal. If not, the script fails and the spend is rejected.
`OP_CHECKSIG (0xAC)`	Checks the signature is valid for this transaction and this public key.

In your code: `Script([0x76, 0xA9, target_public_key, 0x88, 0xAC])`.

Q15 How is a transaction serialized and how is the TxID made?

Say this in the viva: Serialization writes the transaction as bytes in a fixed order. The TxID is the hash of those bytes, byte-reversed for display.

Explained simply

Part	Size
version	4 bytes, little-endian
number of inputs	varint
each input: prev_tx (reversed), prev_index, script_sig, sequence	32 + 4 + varint + script + 4 bytes
number of outputs	varint
each output: amount, script_pubkey	8 bytes + varint + script
locktime	4 bytes, little-endian

A **varint** is a compact length: values below 253 use one byte; bigger values use a marker byte (0xfd, 0xfe or 0xff) plus 2, 4 or 8 bytes. The byte reversal `[::-1]` is a Bitcoin habit: the hash is stored in one byte order but shown in the other. All nodes must serialize identically, or they would calculate different TxIDs for the same transaction.

Q16 What is bytes_needed for in the coinbase script? Why little-endian?

Say this in the viva: bytes_needed finds how many bytes the block height needs, so the height is stored in the shortest possible form. Little-endian is the byte order Bitcoin uses.

Explained simply

Example: height 1 needs 1 byte; height 300 needs 2 bytes. Little-endian means the least important byte comes first (300 = 0x012C is stored as 2C 01). **Small weakness:** the function uses floating-point `log()`, which can give a wrong answer at exact powers of 256. A safer way is `(n.bit_length() + 7) // 8`.

D. Code-level "gotcha" questions

Q17 hash256 is a single SHA-256 but Bitcoin uses double SHA-256. Is the name misleading?

Say this in the viva: Yes. In Bitcoin hash256 means SHA-256 applied twice. Here it is applied once, so either rename it or double-hash it.

Explained simply

Bitcoin's `hash256(x) = sha256(sha256(x))`. Double hashing is an extra safety step against length-extension attacks on plain SHA-256. Fix:

```
def hash256_bytes(data):
    return hashlib.sha256(hashlib.sha256(data).digest()).digest()
```

If you change it, every old hash and the saved chain become invalid, so delete the old Data file and mine again.

Q18 In BlockHeader.mine the nonce is incremented after hashing. What is stored?

Say this in the viva: The saved nonce is one more than the nonce that produced the saved hash, so recalculating the hash from the saved block gives a different result.

Explained simply

Look at the order inside the loop: (1) hash with nonce = 100, (2) **then** nonce becomes 101, (3) check the condition. If the hash from step 1 is good, the loop stops, but the header already says nonce = 101. When someone verifies the block with 101, the hash differs.

Fix: only increase the nonce when the hash is not good:

```
def mine(self, target):
    while True:
        self.blockHash = hash256(self.header_bytes())
        if self.blockHash.startswith(target):
            break               # nonce stays the one that worked
        self.nonce += 1
```

Q19 Tx.to_dict() changes the object itself. What happens if you call it twice?

Say this in the viva: The second call crashes, because the first call replaced the Tx objects inside the transaction with plain dictionaries.

Explained simply

After the first call, `self.tx_ins[0]` is a **dict**, not a TxIn object. On the second call `is_coinbase()` tries `tx_ins[0].prev_tx` and raises **AttributeError**. The transaction can also no longer be serialized or hashed.

This is called a **side effect**. A conversion function should build and return a new dictionary and leave the original object unchanged (for example, work on `copy.deepcopy(self)`).

Q20 What are the problems with your file database?

Say this in the viva: It rewrites the whole file for every block, uses a relative path, has no locking and no validation. A real database such as SQLite or LevelDB would fix this.

Explained simply

Problem	Why it matters	Better approach
Reads and rewrites the entire JSON file for each block	Gets slower as the chain grows	SQLite / LevelDB, or append one block per line
Relative path `"Data"`	Different folder = different (or missing) database	Build an absolute path with `os.path.dirname(__file__)`
No file locking	Two writers at once can corrupt or lose data	A database with transactions
No validation before saving	Bad blocks can be saved	Check hash, prev hash and proof of work first
`lastBlock()` returns None if empty	`add_next_block` would crash	Handle the empty case

Part 4. Bugs and weak points examiners may find

If an examiner opens your code and asks "what would you improve?", pick from this list. Naming problems yourself looks much better than being caught.

Where	Problem	Simple fix
`blockheader.py` mine()	Nonce increased after hashing, so saved nonce does not match saved hash.	Check first, increase only if the hash is not valid (Q18).
`util.py` hash256	Single SHA-256 although the name means double.	Double hash it or rename it (Q17).
`script.py` serialize()	A data push of exactly 75 bytes matches no branch (`< 75`, `> 75`) and raises ValueError.	Use `length <= 75` for the first branch.
`tx.py` to_dict()	Changes the transaction in place; second call crashes.	Return a new dict (Q19).
`tx.py` PRIVATE_KEY	A real private key is written in the source code.	Remove it. Load from a wallet file or environment variable.
`config.py` CURRENT_REWARD	Reward is always 50, never halves.	Reward = 50 >> (height // 210000) (task 4).
`blockchain.py` merkle_root	Set to an empty string.	Compute it from the transaction IDs (task 1).
`blockchain.py` Txs	Saves `Txs[0].to_dict()`, a single dict, not a list.	Save `[tx.to_dict() for tx in Txs]`.
`database.py` path	Relative `"Data"` depends on the current folder.	Use an absolute path (task 10).
`main.py`, `account.py`, `config.py`	Hard-coded Windows paths (`C:/Users/User/...`, `D:\\...`).	Remove them. Run from the project folder instead.
`EllepticCurve/op.py`	Imports `Backend.util.util` (lowercase) but the folder is `Util`. Fails on Linux and macOS.	Use the exact folder names. Also the file is unfinished.
`account.py`	Prints the private key; `randbits` may give an invalid key.	Do not print secrets; use `secrets.randbelow` (Q13).
`Data/uapcoin-blockchain`	The saved file has older test blocks (text transactions) that do not match the current block format.	Delete the file and let the program create a fresh chain.

Part 5. Ten coding tasks with working code

Write these yourself first. Use the code only to check. In the test you will usually get 20 to 40 minutes, so practise the small ones (tasks 1 to 6) until you can do them without looking.

Task 1 Merkle root

Goal: Turn a list of transaction IDs into one hash.

Edit: `blockchain.py` (add a function, then use it in `add_block`).

- If the list is empty return an empty string.
- If the count is odd, repeat the last hash.
- Hash neighbouring pairs until one hash is left.

```
def compute_merkle_root(tx_ids):
    if not tx_ids:
        return ""
    level = list(tx_ids)
    while len(level) > 1:
        if len(level) % 2 == 1:
            level.append(level[-1])              # odd count: duplicate the last
        level = [hash256((level[i] + level[i + 1]).encode())
                 for i in range(0, len(level), 2)]
    return level[0]

# in add_block, replace  merkle_root = ''  with:
merkle = compute_merkle_root([coinbase_tx.TxId])
block_header = BlockHeader(version, prevBlockHash, merkle, timestamp, bits)
```

Task 2 Fix the nonce bug and add verify()

Goal: Make the saved nonce match the saved hash, and be able to re-check a header.

Edit: `blockheader.py`.

- Put the header text in one helper so mining and verifying use the same formula.
- Increase the nonce only when the hash is not valid.

```
class BlockHeader:
    def header_bytes(self):
        return (str(self.version) + self.prev_block_hash + self.merkle_root +
                str(self.timestamp) + str(self.bits) + str(self.nonce)).encode("utf-8")

    def mine(self, target):
        while True:
            self.blockHash = hash256(self.header_bytes())
            if self.blockHash.startswith(target):
                break
            self.nonce += 1

    def verify(self, target):
        return (hash256(self.header_bytes()) == self.blockHash
                and self.blockHash.startswith(target))
```

Task 3 Validate the whole chain

Goal: Return True only if every block is correct and linked properly.

Edit: `Blockchain` class. (Do task 2 first, otherwise the nonce bug makes every block fail.)

- Recompute each block's hash and compare.
- Check it starts with TARGET.
- Check each block's prev_block_hash equals the previous block's hash.
- Test it: edit one value in the data file by hand and show the function returns False.

```
def is_chain_valid(self):
    chain = self.read_from_disk()
    for i, block in enumerate(chain):
        h = block["BlockHeader"]
        text = (str(h["version"]) + h["prev_block_hash"] + h["merkle_root"] +
                str(h["timestamp"]) + str(h["bits"]) + str(h["nonce"]))
        if hash256(text.encode()) != h["blockHash"]:
            return False                              # block was changed
        if not h["blockHash"].startswith(TARGET):
            return False                              # proof of work missing
        if i > 0 and h["prev_block_hash"] != chain[i - 1]["BlockHeader"]["blockHash"]:
            return False                              # link is broken
    return True
```

Task 4 Block reward halving

Goal: Reward halves every 210,000 blocks.

Edit: `config.py` and `CoinbaseTx` in `tx.py` (pass the height in and use the function).

- Right shift by 1 divides by 2, so `>> halvings` does the halving.
- After 64 halvings the reward is 0.

```
HALVING_INTERVAL = 210_000

def block_reward(height):                     # returns poisha
    halvings = height // HALVING_INTERVAL
    if halvings >= 64:
        return 0
    return (INITIAL_REWARD * ONE_COIN) >> halvings

# block_reward(1) -> 5_000_000_000    block_reward(210_000) -> 2_500_000_000
```

Task 5 Numeric target and difficulty adjustment

Goal: Compare the hash as a number and change difficulty if blocks come too fast or too slow.

Edit: `blockheader.py`, `blockchain.py`, `config.py`.

- A hash is valid when `int(hash, 16) < target`.
- Fewer zeros = bigger target = easier. More zeros = harder.
- Every N blocks compare the real time taken with the expected time.

```
def target_from_zeros(zeros):                 # zeros = number of leading hex zeros
    return 1 << (256 - 4 * zeros)

def is_valid(block_hash, zeros):
    return int(block_hash, 16) < target_from_zeros(zeros)

def adjust_zeros(zeros, first_ts, last_ts, n_blocks, seconds_per_block=10):
    actual = last_ts - first_ts
    expected = n_blocks * seconds_per_block
    if actual < expected / 2:
        return zeros + 1                      # too fast: make it harder
    if actual > expected * 2:
        return max(1, zeros - 1)              # too slow: make it easier
    return zeros
```

Task 6 Block lookup and a small command line

Goal: Find a block by height or by hash, and print the last N blocks.

Edit: `blockchain.py` and `main.py`.

- Read the chain, loop through it, compare the field.

```
def get_block_by_height(self, height):
    for block in self.read_from_disk():
        if block["Height"] == height:
            return block

def get_block_by_hash(self, block_hash):
    for block in self.read_from_disk():
        if block["BlockHeader"]["blockHash"] == block_hash:
            return block

# main.py
import argparse, json
p = argparse.ArgumentParser()
p.add_argument("--height", type=int)
p.add_argument("--last", type=int)
a = p.parse_args()
bc = Blockchain()
if a.height is not None:
    print(json.dumps(bc.get_block_by_height(a.height), indent=2))
elif a.last:
    print(json.dumps(bc.read_from_disk()[-a.last:], indent=2))
```

Task 7 Balance of an address

Goal: Add up all coins sent to an address.

Edit: `blockchain.py`.

- Decode the address to its 20-byte hash and turn it into hex.
- Each output has `script_pubkey["cmds"][2]` = the owner hash (hex).
- Sum the matching amounts. **Extension:** to be a true UTXO balance, also remember which outputs were spent (an input's prev_tx and prev_index) and skip them.

```
def get_balance(self, address):
    h160 = decode_base58(address).hex()
    total = 0
    for block in self.read_from_disk():
        tx = block["Txs"]                          # later: for tx in block["Txs"]
        for out in tx["tx_outs"]:
            if out["script_pubkey"]["cmds"][2] == h160:
                total += out["amount"]
    return total / ONE_COIN                         # show in coins
```

Task 8 Create, sign and verify a transaction

Goal: Spend an earlier output: pay someone and send the change back to yourself, then sign it.

Edit: a new function (for example in `tx.py`). Uses `PrivateKey` from `EllepticCurve.py`.

- Pick the unspent output to spend (its TxID and index).
- Outputs: payment + change. Change = input amount - payment - fee.
- Hash the serialized transaction, sign the hash with the private key, put signature + public key in the input's script_sig.
- This is a simplified signing hash. Real Bitcoin also inserts the previous output's script before hashing.

```
pk = PrivateKey(secret_int)                                 # your private key
tx = Tx(1, [TxIn(prev_tx_bytes, prev_index)], [TxOut(pay_amount, pay_script),
                                               TxOut(change, my_script)], 0)

z = int.from_bytes(hash256_bytes(tx.serialize()), "big")    # what we sign
sig = pk.sign(z)
tx.tx_ins[0].script_sig = Script([sig.der() + b"\x01", pk.point.sec()])

assert pk.point.verify(z, sig)                              # must be True
```

Task 9 Mempool and blocks with several transactions

Goal: Keep waiting transactions in a list, check them, and mine them together with the reward.

Edit: `blockchain.py`.

- Keep a list `mempool`. Accept a transaction only if its inputs exist and outputs do not exceed inputs.
- Fee = inputs - outputs. The miner's coinbase = reward + total fees.
- Save `Txs` as a **list**, and compute the Merkle root from all TxIDs.

```
def mine_block(self, mempool, max_tx=10):
    chosen = mempool[:max_tx]
    fees = sum(tx.fee for tx in chosen)                  # fee = inputs - outputs
    coinbase = CoinbaseTx(height, extra_reward=fees).CoinbaseTransaction()
    txs = [coinbase] + chosen
    root = compute_merkle_root([t.TxId for t in txs])
    # build the header with root, call mine(TARGET), then store:
    #   Block(height, size, header_dict, [t.to_dict() for t in txs], len(txs))
    del mempool[:max_tx]
```

Task 10 Wallet file and cleanup

Goal: Save keys safely, fix paths and fix two small bugs.

Edit: `account.py`, `database.py`, `script.py`, `tx.py`.

- Wallet: write keys to a file that is never committed to git.
- Database path: build it from the file location.
- Script: allow 75-byte pushes. to_dict: do not change the original.

```
# wallet
json.dump({"private_key": hex(self.privateKey), "address": self.PublicAddress},
          open("wallet.json", "w"))

# database.py : path that works from any folder (this file is in Blockchain/Backend/Core/Database)
HERE = os.path.dirname(os.path.abspath(__file__))
self.basepath = os.path.join(HERE, "..", "..", "..", "Data")
self.filepath = os.path.join(self.basepath, self.filename)

# script.py : include 75
if length <= 75:
    result += int_to_little_endian(length, 1)

# tx.py : to_dict must not change the object
import copy
def to_dict(self):
    tx = copy.deepcopy(self)
    ...                     # do the same conversions on tx, then return tx.__dict__
```

Part 6. Last-minute tips

- **Explain the flow, not the lines.** "Wallet makes an address, the miner builds a block with a reward transaction, mining finds a hash with four zeros, and the block is saved." That one sentence answers many questions.
- **Use the cash-note example** for UTXO and the "seal on a page" idea for hash links. Simple pictures impress more than long definitions.
- **Be honest about what is missing** (Merkle root, chain validation, normal transactions, network). Then say how you would add it. Examiners prefer this to guessing.
- **Know your numbers:** target 0000, reward 50, 1 coin = 100,000,000 poisha, private key 256 bits, hash160 = 20 bytes, address checksum = 4 bytes.
- **In the coding test:** read the data file format first (open `Data/uapcoin-blockchain`), write the smallest working version, run it, then improve it. Test with a tampered block or a wrong address.
- **Keep your environment ready:** Python installed, `pycryptodome` installed, the project runs from the `Blockchain` folder, and the old Data file deleted if the format changed.